

Question Bank — Week 6

Stacks (Lecture 06) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week06-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. *Stack* is the class of `dsa/stack.py`: `push`, `pop`, `peek`, `is_empty`, `Len`, with `pop` and `peek` raising `IndexError` on an empty stack. Operators in expressions are `+` `-` `*` `/` with the usual precedence, all left associative.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W6-M01 – W6-M22
B	Short answer and essay	W6-E1 – W6-E5
C	Trace the code and the algorithms	W6-T1 – W6-T5
D	Find and fix the bug	W6-B1 – W6-B4
E	Write the code — checked by <code>pytest</code>	W6-C1 – W6-C5

Part A — Multiple choice

W6-M01 [what] "LIFO" means:

- **a)** the last item pushed is the first one popped
- **b)** the first item pushed is the first one popped
- **c)** items are popped in sorted order
- **d)** any item can be popped

W6-M02 [how] `push(1)`, `push(2)`, `push(3)`, `pop()`, `push(4)`, `pop()`. What does the **next** `pop()` return?

- **a)** 4
- **b)** 3
- **c)** 2
- **d)** 1

W6-M03 [why] A stack built on a dynamic array should keep its top at:

- **a)** index 0
- **b)** the end — the last used slot
- **c)** the middle

- **d)** it makes no difference

W6-M04 [why] A stack built on a singly linked list should keep its top at:

- **a)** the last node
- **b)** the middle node
- **c)** a separate array
- **d)** the head

W6-M05 [what] `Stack().pop()` on an empty stack:

- **a)** raises `IndexError`
- **b)** returns `None`
- **c)** returns `0`
- **d)** blocks until something is pushed

W6-M06 [why] A stack on an array with its top at **index 0**. push costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(n)$ — every element shifts
- **d)** $O(\log n)$

W6-M07 [how] Why is "`( [ ] )`" not balanced?

- **a)** It has an odd number of brackets
- **b)** `)` arrives when the most recent opener is `[`
- **c)** The stack is not empty at the end
- **d)** A closer arrives when the stack is empty

W6-M08 [how] Which check makes `is_balanced("")` return `False`?

- **a)** the partner check
- **b)** the empty-at-the-end check
- **c)** a length check
- **d)** a closer arrives when the stack is empty

W6-M09 [how] The postfix form of `3 + 4 * 2` is:

- **a)** `3 4 2 * +`
- **b)** `3 4 + 2 *`
- **c)** `+ 3 * 4 2`
- **d)** `3 + 4 2 *`

W6-M10 [how] The postfix form of `( 3 + 4 ) * 2` is:

- **a)** `3 4 2 * +`
- **b)** `3 + 4 * 2`
- **c)** `3 4 + 2 *`
- **d)** `* + 3 4 2`

W6-M11 [how] Evaluating the postfix `8 3 -` gives:

- **a)** `-5`

- **b)** 5
- **c)** 11
- **d)** an error

W6-M12 [how] Evaluating the postfix $2\ 3\ 4\ *\ +$ gives:

- **a)** 20
- **b)** 9
- **c)** 24
- **d)** 14

W6-M13 [what] In the shunting-yard algorithm, when an operator arrives, it first pops to the output every stacked operator with:

- **a)** higher or equal precedence
- **b)** lower precedence
- **c)** any precedence
- **d)** strictly higher precedence only

W6-M14 [why] Why does an operator of **equal** precedence also leave the stack?

- **a)** It is faster
- **b)** To keep the stack small
- **c)** So that $-$ and $/$ are left associative: $8 - 3 - 2$ is $(8 - 3) - 2$
- **d)** It makes no difference to the result

W6-M15 [how] The postfix form of $8 - 3 - 2$ is:

- **a)** $8\ 3\ 2\ -\ -$
- **b)** $8\ 3\ -\ 2\ -$
- **c)** $- \ -\ 8\ 3\ 2$
- **d)** $8\ -\ 3\ 2\ -$

W6-M16 [how] The worst-case extra space of `is_balanced` on a text of length n is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(n)$

W6-M17 [why] To turn a recursive algorithm into a loop, in general you need:

- **a)** an explicit stack
- **b)** a queue
- **c)** a sorted array
- **d)** nothing — every recursion is already a loop

W6-M18 [what] In an undo/redo system with two stacks, what happens to the redo stack when the user performs a **new** action?

- **a)** the new action is pushed on it
- **b)** it is cleared
- **c)** its top is undone

- **d)** nothing

W6-M19 [how] n pushes onto the Stack of `dsa/stack.py` (built on a `DynamicArray`) cost in total:

- **a)** $O(n^2)$
- **b)** $O(n \log n)$
- **c)** $O(n)$ — amortised $O(1)$ each
- **d)** $O(1)$

W6-M20 [what] Which of these is **not** a natural stack application?

- **a)** checking balanced brackets
- **b)** evaluating postfix expressions
- **c)** undo in an editor
- **d)** serving customers in the order they arrived

W6-M21 [how] While `is_balanced("((((())))")` runs, the largest number of items on the stack at once is:

- **a)** 4
- **b)** 8
- **c)** 1
- **d)** 2

W6-M22 [what] `peek()` differs from `pop()` in that it:

- **a)** is slower
 - **b)** returns the top without removing it
 - **c)** returns the bottom
 - **d)** never raises an error
-

Part B — Short answer and essay

W6-E1 [why] (4 marks) State the Stack ADT with the cost of each operation. Describe two honest implementations, compare them, and describe one design that passes every test but is not honest.

W6-E2 [how] (4 marks) Describe the stack algorithm for balanced brackets. Give an example of each of the three ways a text can be unbalanced, and the check that catches it.

W6-E3 [how] (3 marks) Explain how a stack evaluates a postfix expression. Why does the order in which the two operands are popped matter, and for which operators?

W6-E4 [why] (4 marks) Explain the shunting-yard algorithm, and trace it on `2 * ( 3 + 4 ) - 5`. Why must an operator of equal precedence be popped?

W6-E5 [why] (3 marks) "Recursion is a stack." Explain, and describe how you would rewrite a recursive algorithm without recursion. When is that worth doing in Python?

Part C — Trace the code and the algorithms

W6-T1 [how] What is printed?

```
s = Stack()
out = []
s.push(1); s.push(2)
out.append(s.pop())
s.push(3); s.push(4)
out.append(s.peek())
out.append(s.pop())
out.append(s.pop())
out.append(len(s))
print(out)
```

W6-T2 [how] Evaluate the postfix expression `5 1 2 + 4 * + 3 -`, showing the stack after every token.

W6-T3 [how] Convert `2 * ( 3 + 4 ) - 5` to postfix with the shunting-yard algorithm, showing the output and the operator stack after every token.

W6-T4 [how] Run `is_balanced("{[()()]})"` by hand. Show the stack after every character. What is the maximum stack size?

W6-T5 [why] In what order are the numbers printed? Why that order?

```
pending = Stack([[1, [2, 3], 4]])
while not pending.is_empty():
    item = pending.pop()
    if isinstance(item, list):
        for inner in item:
            pending.push(inner)
    else:
        print(item, end=" ")
```

Part D — Find and fix the bug

W6-B1 [how]

```
def pop(self):
    if len(self._items) == 0:
        return None
    return self._items.pop()
```

W6-B2 [how]

```
def is_balanced(text):
    pairs = {"(": ")", "[": "]", "{": "}"
    s = Stack()
    for ch in text:
        if ch in "([{":
            s.push(ch)
        elif ch in pairs:
            if s.is_empty() or s.pop() != pairs[ch]:
                return False
    return True
```

W6-B3 [how] Part of `evaluate_postfix`:

```
if token in "+-*/":
    a = stack.pop()
    b = stack.pop()
    stack.push(apply(token, a, b))    # apply(op, left, right)
```

W6-B4 [why] Part of `infix_to_postfix`:

```
while not ops.is_empty() and ops.peek() in prec and prec[ops.peek()] > prec[token]:
    output.append(ops.pop())
ops.push(token)
```

Part E — Write the code

In `practice/week06.py`; check with `pytest tests/test_practice_week06.py -v`. Use your `Stack` as working storage.

W6-C1 [how] `reverse_words(sentence)`.

W6-C2 [why] `MinStack` — `push`, `pop`, `peek` and `get_min`, all $O(1)$. Why is keeping a single “current minimum” variable not enough?

W6-C3 [why] `next_greater(values)` in $O(n)$. Why is the total work linear, even though there is a loop inside a loop?

W6-C4 [how] `tags_balanced(html)` — the brackets algorithm, for HTML tags.

W6-C5 [how] `decode(encoded)` — expand `k[text]`, which may nest.